Intermediate | AI Agent Security | Highlights | IBM Technology

Guide to Architect Secure AI Agents: Best Practices for Safety

Understand the threats autonomous AI agents introduce and the architecture controls that keep them safe, governed, and auditable.

For software architects, security engineers, and technical leaders building or governing autonomous AI agents · 27 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

Watch the source video | Read the full lesson | Read the condensed version

What matters

- ✓ Agents are probabilistic and adaptive, unlike traditional deterministic software, so identical inputs can produce different outputs and behavior can change over time based on feedback.
- ✓ Agents widen the attack surface in two specific places: the AI model itself, and the MCP protocol or equivalent channel the agent uses to reach tools and services.
- ✓ Prompt injection is the top attack against LLM-based systems: hidden instructions embedded in content the agent reads can hijack its behavior as if an attacker had remote control.
- ✓ Because an agent acts autonomously, a compromised agent becomes an attack amplifier, executing malicious actions at machine speed faster than a human could intervene.
- ✓ The principle of least privilege, enforced through role-based access control (RBAC) and sandboxing, limits how much damage a compromised or misbehaving agent can do.
- ✓ Agents need their own non-human identities with unique, short-lived, just-in-time credentials so that every action can be traced back to the specific agent that performed it.
- ✓ Routing agent traffic through an AI gateway (a firewall or proxy in front of the model and MCP calls) lets you inspect requests for prompt injection and inspect responses for data leakage before either lands.
- ✓ Security has to be continuous: real-time threat detection, proactive threat hunting, and monitoring for configuration or model drift, not a one-time check before deployment.

What it covers

01 **The paradigm shift: from deterministic code to probabilistic agents** `1:28`
Agents differ from traditional software along three axes: they are probabilistic instead of deterministic, adaptive instead of static, and built around continuous evaluation instead of a fixed implementation.

02 **The agent development lifecycle and DevSecOps** `2:13`
Agents need a structured lifecycle — plan, code, test, debug, deploy, monitor, and back to plan — with security integrated at every stage rather than bolted on at the end.

03 **How agents expand the attack surface** `3:37`
Agents introduce two new attack surfaces — the AI model and the protocol connecting it to tools — plus new risk categories that come specifically from acting autonomously.

04 **System controls and design principles for safe agents** `5:49`
Three system controls — constrained, permissioned, and sandboxed — combine with design principles like acceptable agency, secure-by-design, and least privilege to bound what an agent can do.

05 **Identity and access management for non-human identities** `7:57`

Agents need their own unique credentials, just-in-time access, and role assignments — the same identity discipline used for human users — so every action can be traced back to a specific agent.

06 **Securing data and the model with an AI gateway** `9:22`

Instead of letting requests hit the model or MCP tools directly, route them through a gateway or proxy that enforces policy, screens for prompt injection, and checks for data loss before either side of the conversation completes.

07 **Continuous threat detection, hunting, and risk assessment** `10:46`

Once deployed, an agent needs real-time monitoring for abnormal behavior, proactive threat hunting, ongoing risk assessment of what it's capable of, and drift monitoring across its whole lifecycle.

Glossary

MCP (Model Context Protocol) — A protocol that lets an AI agent connect to and call external tools and services in a standardized way. It is also a new attack surface, since anything the agent can reach through MCP is something an attacker might reach too.

RBAC (Role-Based Access Control) — An access model where permissions are granted through assigned roles rather than to individuals directly, so an agent's allowed actions are defined by the role it holds.

DevSecOps — A development discipline that integrates security practices throughout the entire software lifecycle — planning, building, testing, deploying, and monitoring — instead of adding security only near release.

Just-in-time (JIT) access — Granting a permission only for the duration a task actually needs it, often with an explicit expiration, rather than issuing a standing credential that persists indefinitely.

Excessive agency — A condition where an agent has been granted more autonomy, access, or tool capability than its task actually requires, increasing the potential damage from any mistake or attack.

Prompt injection — An attack where instructions hidden inside content an agent processes (a document, webpage, or tool response) get treated as legitimate commands, effectively giving an attacker remote control over the agent's behavior.

Least privilege — The principle that a system or person should have access to only what is needed to complete the current task, with that access removed as soon as it is no longer needed.

Non-human identity — A unique, individually credentialed identity assigned to a software system such as an AI agent, analogous to a user account for a person, so actions can be traced back to the specific system that performed them.

AI gateway — A proxy or firewall placed between an agent and the model or tools it calls, used to enforce policy, screen for prompt injection on the way in, and check for data loss on the way out.

Configuration drift — Gradual, often unnoticed change in a system's operating parameters over time, which for a self-modifying agent can happen without any explicit code change or deployment.